Modules

CommonJS (old) and ES Modules (new) Styles

Node.js CommonJS Modules

Node.js originally used the **CommonJS** module system, before ES Modules (`import/export`) were introduced.

We use:

- `require()` to **import**
- `module.exports` to **export**

Also:

- Adding the ".js" extension is optional.

Class Module Example

lib/Calculator.js

```
class Calculator {
  add(a, b) {
    return Number(a) + Number(b);
  }
  multiply(a, b) {
    return Number(a) * Number(b);
  }
}

module.exports = Calculator;
```

Exporting a class using `module.exports = Calculator`.

require to use the module

```
// Demo consumer using require(...)
const Calculator = require('./lib/Calculator');

const calc = new Calculator();

const a = 12;
const b = 3;

console.log('Using Calculator class:');
console.log(`add(${a}, ${b}) =`, calc.add(a, b));
console.log(`multiply(${a}, ${b}) =`, calc.multiply(a, b));
```

Function Module Example

lib/mathFns.js

```
function sub(a, b) {
  return Number(a) - Number(b);
}

function div(a, b) {
  if (Number(b) === 0) throw new Error("Division by zero");
  return Number(a) / Number(b);
}

module.exports = { sub, div };
```

Exporting multiple functions as an object.

require to use the module

```
// object destructuring assignment
const { sub, div } = require('./lib/mathFns');

const a = 12;
const b = 3;

console.log(`sub(${a}, ${b}) =`, sub(a, b));
console.log(`div(${a}, ${b}) =`, div(a, b));
```

We also can use:

```
const arith = require('./lib/mathFns');
console.log(`sub(${a}, ${b}) =`, arith.sub(a, b));
console.log(`div(${a}, ${b}) =`, arith.div(a, b));
```

Node.js ES Modules (Modern Syntax)

In modern Node.js (v14+), we use **ES Modules (ESM)** instead of CommonJS.

We use:

- `import` to **import**
- `export` to **export**

We can also specify a default export.

Also:

- ESM is enabled with `"type": "module"` in `package.json` (or with the `.mjs` extension).
- We must include the ".js" extension when we specify the module name.

Class Module Example

```
// Named exports
export class Calculator {
  add(a, b) {
    return Number(a) + Number(b);
  }
  multiply(a, b) {
    return Number(a) * Number(b);
  }
}
```

Exporting a class using `export class`.

import to use the module

```
// Demo consumer using import
import { Calculator } from './math.js';

const calc = new Calculator();

const a = 12;
const b = 3;

console.log('Using Calculator class:');
console.log(`add(${a}, ${b}) =`, calc.add(a, b));
console.log(`multiply(${a}, ${b}) =`, calc.multiply(a, b));
```

Function Module Example

```
// Default export
export default function sub(a, b) {
  return Number(a) - Number(b);
}

// Named exports
export function add(a, b) {
  return Number(a) + Number(b);
}

export function div(a, b) {
  if (Number(b) === 0) throw new Error("Division by zero");
  return Number(a) / Number(b);
}
```

Here we export:

- sub as the default export
- add and div as named exports

import to use the module

```
// import default and named exports
import sub, { div } from './math.js';

const a = 12; const b = 3;

console.log(`sub(${a}, ${b}) =`, sub(a, b));
console.log(`div(${a}, ${b}) =`, div(a, b));
```

In this example:

- sub ← comes from export default …
- { div } ← comes from export function div(…)

They come from two different “export namespaces”:

- The default export has no name.
- The named exports are grouped inside { … }.

import all exports

We can use `* as name` to access all the exports using the `name` (we can use any name for it).

```
import sub, * as math from './math.js';

const a = 12;
const b = 3;

console.log('Using ES Module Functions:');
console.log(`add(${a}, ${b}) =`, math.add(a, b));
```

import CommonJS module

When we use a CommonJS module in an ES module, the default import gives us the whole `module.exports` object of the CommonJS module, so we assign it to a variable.

```
// lib/mathFns.cjs is a CommonJS module: module.exports = { sub, div };
import mathFns from './lib/mathFns.cjs';

console.log(`sub(${a}, ${b}) =`, mathFns.sub(a, b));
```

The `.cjs` extension tells Node.js that the file is CommonJS, even in a project with `"type": "module"`.